Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)
КАФЕДРА «Информационная безопасность» (ИУ8)

ЛАБОРАТОРНАЯ РАБОТА

«Построение линейного классификатора Фишера»

по дисциплине «Введение в искусственный интеллект»

Вариант 2

Студент ИУ8-13М (Группа) Бочкарев Михаил Александрович (И. О. Фамилия) _____ (Подпись, дата)

Преподаватель Басараб Михаил Алексеевич (И. О. Фамилия) _____ (Подпись, дата)

Оценка: _____

Москва, 2026 г.

СОДЕРЖАНИЕ

1 Цель работы 2
ВВЕДЕНИЕ 3
2 Исходные данные и постановка задачи 4
3 Геометрическая модель 5
4 Контрольный метод: пороговый перцептрон 6
5 Линейный классификатор Фишера 7
5.1 Центроиды классов 7
5.2 Матрицы разброса 7
5.3 Опорное направление и нормировка 8
5.4 Проецирование вершин и расчёт разбросов 8
5.5 Порог и дискриминантная гиперплоскость 9
6 Сравнение построенных разделителей 11
6.1 Граничные рёбра 11
ЗАКЛЮЧЕНИЕ 13
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 14
ПРИЛОЖЕНИЕ А Листинг программы 15

1 Цель работы

Построить оптимальную по критерию Фишера дискриминантную гиперплоскость для линейно разделимой модели бинарной классификации «Свой–Чужой» на четырёхмерном гиперкубе для варианта 2.

ВВЕДЕНИЕ

Исследуется бинарная классификация типа «Свой–Чужой», заданная булевой функцией четырёх переменных на вершинах четырёхмерного гиперкуба. Согласно индивидуальному варианту 2 методических указаний, моделируемая функция имеет вид

$$F(x_1, x_2, x_3, x_4) = (\overline{x_3} + x_4)\overline{x_1} + x_2, \tag{1}$$

где сложение обозначает логическое ИЛИ, умножение — логическое И, черта — отрицание. Это то же самое выражение, которое исследовалось при подготовке программы. В отличие от предварительного сводного отчёта теперь использованы официальные формулировки задач: метод Фишера и линейное программирование симплекс-методом, а не метод опорных векторов.

2 Исходные данные и постановка задачи

Вариант 2 соответствует выражению (1). Рассматривается полная таблица истинности из 16 двоичных векторов и граф гиперкуба, содержащий 32 ребра. Граничными считаются рёбра, соединяющие вершины с различными значениями функции. Для функции варианта 2 получено 5 вершин класса 0 и 11 вершин класса 1; программа выделила 10 граничных рёбер. Проверка совместности системы $t_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$ подтверждает строгую линейную разделимость.

Таблица 1 – Таблица истинности булевой функции варианта 2

x_1	x_2	x_3	x_4	F
0	0	0	0	1
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

3 Геометрическая модель

Каждая вершина имеет координаты (x_1, x_2, x_3, x_4), где $x_j \in \{0, 1\}$. Между соседними вершинами проводится ребро тогда и только тогда, когда отличается ровно одна координата. Для визуализации используется линейное отображение $P : \mathbb{R}^4 \to \mathbb{R}^3$:

$$P(\mathbf{x}) = \sum_{j=1}^{4} x_j \mathbf{e}_j, \qquad \begin{aligned} &\mathbf{e}_1 = (1, 0, 0), \quad \mathbf{e}_2 = (0, 1, 0), \\ &\mathbf{e}_3 = (0, 0, 1), \quad \mathbf{e}_4 = (-0{,}4, \sqrt{0{,}03}, 0{,}9). \end{aligned} \tag{2}$$

Все отображаемые направления имеют единичную длину. Вычисления методов ведутся в исходном четырёхмерном пространстве; трёхмерная проекция применяется только к геометрическим сечениям гиперкуба. На рисунках сохранены четыре стрелки-оси, подписи всех 16 вершин и граничные рёбра, выделенные красным.

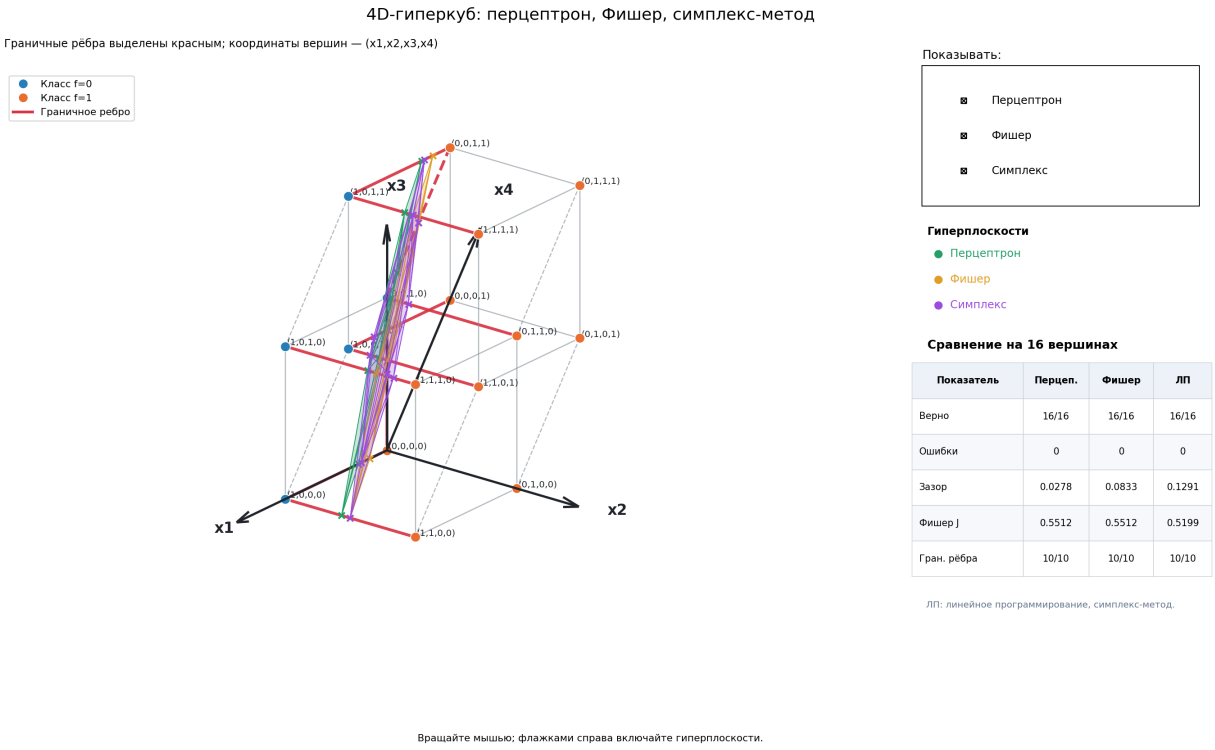


Рисунок 1 – Единая визуализация гиперкуба и трёх разделяющих гиперплоскостей варианта 2

4 Контрольный метод: пороговый перцептрон

Для сопоставления с первой лабораторной применён перцептрон с выходом $y = 1$ при $net \geq 0$ и $y = 0$ при $net < 0$, где $net = \mathbf{w}^T\mathbf{x} + b$. Коррекция производится по формуле

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} + \eta(d - y)\mathbf{x}, \qquad b^{(k+1)} = b^{(k)} + \eta(d - y), \qquad \eta = 0{,}3. \tag{3}$$

Начальные веса равны нулю, начальное смещение $b = -0{,}05$. На полной таблице правильная классификация всех 16 наборов достигнута за 11 эпох. Получено уравнение

$$-0{,}9x_1 + 1{,}5x_2 - 0{,}3x_3 + 0{,}3x_4 + 0{,}25 = 0. \tag{4}$$

5 Линейный классификатор Фишера

5.1 Центроиды классов

По методическим указаниям определены средние векторы:

$$\mathbf{m}_0 = \begin{pmatrix} 0,8000 \\ 0,0000 \\ 0,6000 \\ 0,4000 \end{pmatrix}, \qquad \mathbf{m}_1 = \begin{pmatrix} 0,3636 \\ 0,7273 \\ 0,4545 \\ 0,5455 \end{pmatrix}. \tag{5}$$

5.2 Матрицы разброса

Для $k \in \{0, 1\}$ используем определение $S_k = \sum_{\mathbf{x} \in C_k} (\mathbf{x} - \mathbf{m}_k)(\mathbf{x} - \mathbf{m}_k)^T$. Тогда

$$S_0 = \begin{pmatrix} 0,800 & 0,000 & -0,400 & 0,400 \\ 0,000 & 0,000 & 0,000 & 0,000 \\ -0,400 & 0,000 & 1,200 & -0,200 \\ 0,400 & 0,000 & -0,200 & 1,200 \end{pmatrix}. \tag{6}$$

$$S_1 = \begin{pmatrix} 2,545 & 1,091 & 0,182 & -0,182 \\ 1,091 & 2,182 & 0,364 & -0,364 \\ 0,182 & 0,364 & 2,727 & 0,273 \\ -0,182 & -0,364 & 0,273 & 2,727 \end{pmatrix}. \tag{7}$$

Суммарная матрица разброса и обратная матрица имеют вид

$$S_W = S_0 + S_1 = \begin{pmatrix} 3,345 & 1,091 & -0,218 & 0,218 \\ 1,091 & 2,182 & 0,364 & -0,364 \\ -0,218 & 0,364 & 3,927 & 0,073 \\ 0,218 & -0,364 & 0,073 & 3,927 \end{pmatrix}, \tag{8}$$

$$S_W^{-1} = \begin{pmatrix} 0,368 & -0,197 & 0,039 & -0,039 \\ -0,197 & 0,579 & -0,066 & 0,066 \\ 0,039 & -0,066 & 0,263 & -0,013 \\ -0,039 & 0,066 & -0,013 & 0,263 \end{pmatrix}. \tag{9}$$

Определитель $\det(S_W) = 88,436364$, поэтому обратная матрица существует; регуляризация не потребовалась.

5.3 Опорное направление и нормировка

Вектор линейного дискриминанта определяется по критерию

$$J(\mathbf{w}) = \frac{[\mathbf{w}^T(\mathbf{m}_1 - \mathbf{m}_0)]^2}{\mathbf{w}^T S_W \mathbf{w}} \longrightarrow \max, \quad \mathbf{w}_* = S_W^{-1}(\mathbf{m}_1 - \mathbf{m}_0). \tag{10}$$

Для варианта 2 получено

$$\mathbf{w}_* = \begin{pmatrix} -0,315789 \\ 0,526316 \\ -0,105263 \\ 0,105263 \end{pmatrix}, \qquad \widehat{\mathbf{w}} = \frac{\mathbf{w}_*}{\|\mathbf{w}_*\|} = \begin{pmatrix} -0,500000 \\ 0,833333 \\ -0,166667 \\ 0,166667 \end{pmatrix}. \tag{11}$$

5.4 Проецирование вершин и расчёт разбросов

Выполняем $z_i = \widehat{\mathbf{w}}^T \mathbf{x}_i$. Результаты для обеих групп представлены в таблице.

Таблица 2 – Проекции шестнадцати вершин на направление Фишера

x_1	x_2	x_3	x_4	F	z
0	0	0	0	1	0,00000
0	0	0	1	1	0,16667
0	0	1	0	0	−0,16667
0	0	1	1	1	−0,00000
0	1	0	0	1	0,83333
0	1	0	1	1	1,00000
0	1	1	0	1	0,66667
0	1	1	1	1	0,83333
1	0	0	0	0	−0,50000
1	0	0	1	0	−0,33333
1	0	1	0	0	−0,66667
1	0	1	1	0	−0,50000
1	1	0	0	1	0,33333
1	1	0	1	1	0,50000
1	1	1	0	1	0,16667
1	1	1	1	1	0,33333

Средние проекций составили $\bar{z}_0 = -0,433333$ и $\bar{z}_1 = 0,439394$. Вычисленные показатели:

$$\begin{aligned} D_B &= (\bar{z}_1 - \bar{z}_0)^2 = 0,761653, \\ D_{W0} &= \sum_{z \in C_0} (z - \bar{z}_0)^2 = 0,144444, \\ D_{W1} &= \sum_{z \in C_1} (z - \bar{z}_1)^2 = 1,237374, \\ D_W &= D_{W0} + D_{W1} = 1,381818, \\ J(\widehat{\mathbf{w}}) &= D_B / D_W = 0,551196. \end{aligned} \tag{12}$$

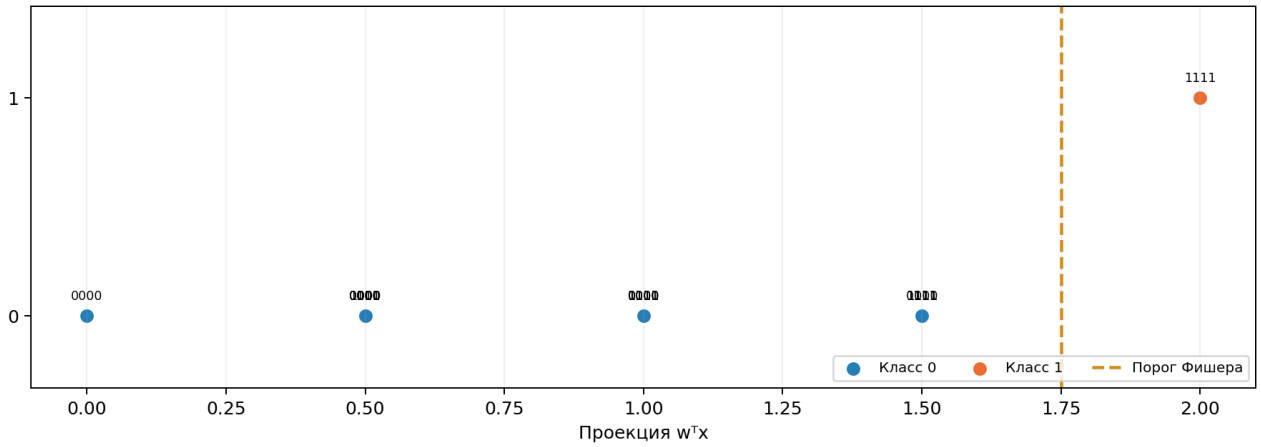


Рисунок 2 – Проекции классов на оптимальное направление Фишера и порог классификации

5.5 Порог и дискриминантная гиперплоскость

В соответствии с пунктом 9 методических указаний порог задаётся **полусуммой крайних проекций классов**, а не подбирается минимизацией числа ошибок:

$$\theta = \frac{\max_{\mathbf{x} \in C_0} (\widehat{\mathbf{w}}^T \mathbf{x}) + \min_{\mathbf{x} \in C_1} (\widehat{\mathbf{w}}^T \mathbf{x})}{2} = \frac{-0,166667 + -0,000000}{2} = -0,083333. \tag{13}$$

Полученное уравнение гиперплоскости $\widehat{\mathbf{w}}^T \mathbf{x} = \theta$:

$$\boxed{-0{,}5x_1 + 0{,}833333x_2 - 0{,}166667x_3 + 0{,}166667x_4 + 0{,}083333 = 0}. \tag{14}$$

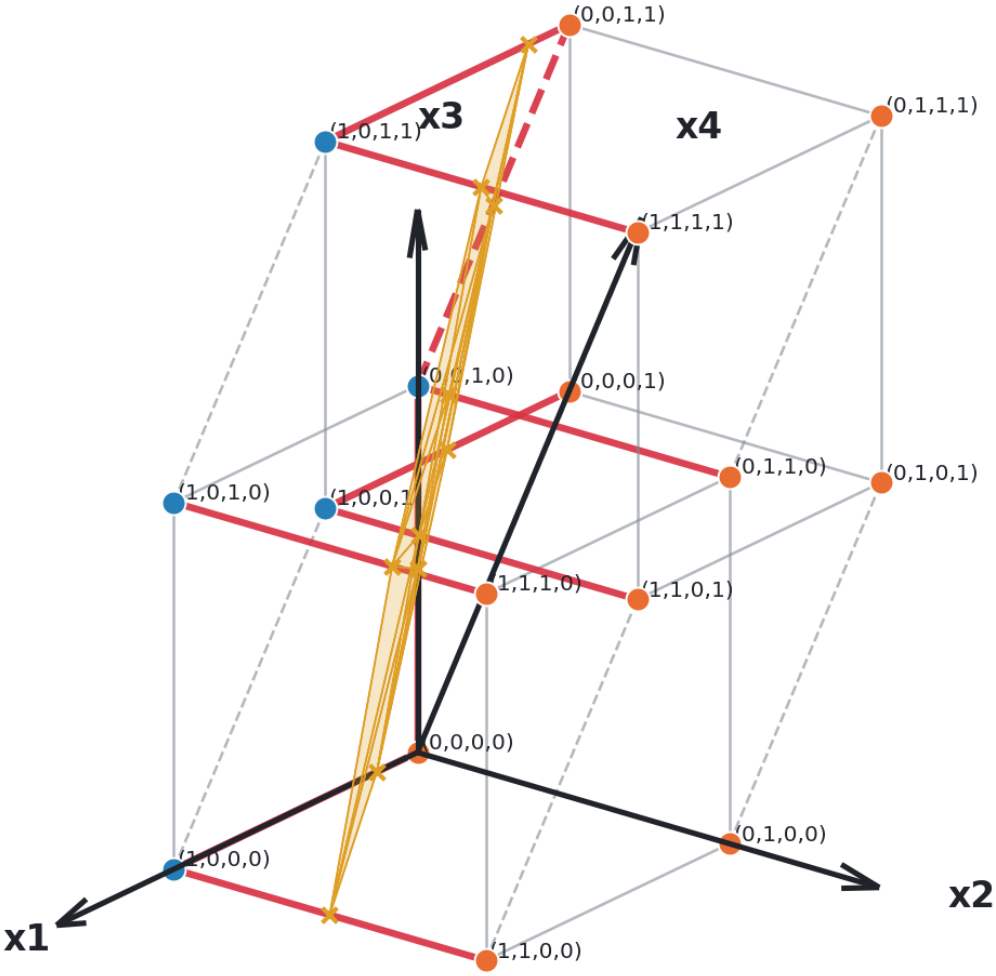


Рисунок 3 – Гиперкуб варианта 2 с сечением, полученным методом Фишера

6 Сравнение построенных разделителей

Таблица 3 – Весовые коэффициенты гиперплоскостей

Метод	w_1	w_2	w_3	w_4	b
Перцептрон	$-0,9000$	$1,5000$	$-0,3000$	$0,3000$	$0,2500$
Фишер	$-0,5000$	$0,8333$	$-0,1667$	$0,1667$	$0,0833$
Симплекс	$-0,8000$	$1,2000$	$-0,4000$	$0,4000$	$0,2000$

Таблица 4 – Показатели классификации на полной таблице истинности

Метод	Верно	Ошибки	γ	J	Рёбра
Перцептрон	16/16	0	0,02778	0,551196	10/10
Фишер	16/16	0	0,08333	0,551196	10/10
Симплекс	16/16	0	0,12910	0,519902	10/10

Критерий J — функционал Фишера; $\gamma = \min_i t_i(\mathbf{w}^T\mathbf{x}_i + b)/\|\mathbf{w}\|$ — минимальный геометрический знаковый зазор. Важно, что методы оптимизируют различные величины: максимальный J не равнозначен максимальному зазору. На рассматриваемых данных направления Фишера и перцептрона совпали (угол 0°), тогда как между направлением Фишера и симплекс-решением угол составляет около $8{,}2055^\circ$.

6.1 Граничные рёбра

На каждом граничном ребре точка пересечения определяется формулой

$$\mathbf{p}(t) = \mathbf{a} + t(\mathbf{b} - \mathbf{a}), \qquad t = -\frac{g(\mathbf{a})}{g(\mathbf{b}) - g(\mathbf{a})}, \quad g(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b. \tag{15}$$

В таблице сначала указывается вершина класса 0, затем вершина класса 1. Параметр $t \in [0, 1]$ задаёт относительное положение пересечения.

Таблица 5 – Точки пересечения с десятью граничными рёбрами

Класс 0	Класс 1	Перцептрон	Фишер	Симплекс
0010	0000	0,167	0,500	0,500
1000	0000	0,722	0,833	0,750
1001	0001	0,389	0,500	0,250
0010	0011	0,167	0,500	0,500
0010	0110	0,033	0,100	0,167
1011	0011	0,722	0,833	0,750
1000	1100	0,433	0,500	0,500
1001	1101	0,233	0,300	0,167
1010	1110	0,633	0,700	0,833
1011	1111	0,433	0,500	0,500

Все три метода пересекают 10 из 10 граничных рёбер, но в разных точках. Совпадение точности 16/16 само по себе не определяет оптимальный разделитель. Во всех расчётах использована только полная таблица истинности без выделения тестовых подвыборок.

ЗАКЛЮЧЕНИЕ

По всем десяти пунктам алгоритма, указанного в методических материалах, получен дискриминант Фишера для варианта 2. Вычислены центроиды, индивидуальные и суммарная матрицы разброса, обратная матрица, нормированный вектор, проекции, функционал $J = 0{,}551196$ и порог $\theta = -0{,}083333$. Гиперплоскость правильно классифицирует все 16 вершин и пересекает десять граничных рёбер. Её направление оптимально по критерию Фишера, но не обязательно по критерию максимального геометрического зазора.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Построение линейного классификатора Фишера. — Методические материалы по лабораторной работе, вариант 2.

2. Линейный дискриминант Фишера. — Учебные материалы по дисциплине «Введение в искусственный интеллект».

ПРИЛОЖЕНИЕ А

Листинг программы

```
import argparse
import ast
import csv
import itertools
import math
from pathlib import Path

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from matplotlib.widgets import CheckButtons
from mpl_toolkits.mplot3d import Axes3D
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
from scipy.optimize import linprog
from scipy.spatial import ConvexHull, QhullError


BASIS = np.array([
    [1.0, 0.0, 0.0],
    [0.0, 1.0, 0.0],
    [0.0, 0.0, 1.0],
    [-0.4, math.sqrt(0.03), 0.9],
], dtype=float)

EXAMPLE_FUNCTION = '(((not x3) or x4) and (not x1)) or x2'
ETA = 0.3
MAX_EPOCHS = 5000
W0_INITIAL = -0.05

COLORS = {
    'Перцептрон': '#23a066',
    'Фишер': '#df9e24',
    'Симплекс': '#9a4ce0',
}
COLOR_ZERO = '#257eb8'
COLOR_ONE = '#e96c31'
COLOR_BOUNDARY = '#d83343'
EPS = 1e-9


def normalize_expression(expression):
    return (expression.strip().replace('¬', ' not ')
            .replace('∧', ' and ').replace('∨', ' or ')
            .replace('⊕', ' ^ ').strip())


def evaluate_node(node, env):
    if isinstance(node, ast.Expression):
```

```
        return evaluate_node(node.body, env)
    if isinstance(node, ast.Name) and node.id in env:
        return bool(env[node.id])
    if isinstance(node, ast.Constant) and type(node.value) in (bool, int):
        if node.value in (0, 1):
            return bool(node.value)
    if isinstance(node, ast.UnaryOp) and isinstance(node.op, (ast.Not, ast.Invert)):
        return not evaluate_node(node.operand, env)
    if isinstance(node, ast.BoolOp):
        vals = [evaluate_node(v, env) for v in node.values]
        if isinstance(node.op, ast.And):
            return all(vals)
        if isinstance(node.op, ast.Or):
            return any(vals)
    if isinstance(node, ast.BinOp):
        a = evaluate_node(node.left, env)
        b = evaluate_node(node.right, env)
        if isinstance(node.op, ast.BitAnd):
            return a and b
        if isinstance(node.op, ast.BitOr):
            return a or b
        if isinstance(node.op, ast.BitXor):
            return a != b
    raise ValueError('Разрешены только x1..x4, 0/1, not/and/or, ~, &, |, ^ и скобки')


def function_labels(expression, X):
    tree = ast.parse(normalize_expression(expression), mode='eval')
    labels = []
    for bits in X:
        env = {'x%d' % (k + 1): int(bits[k]) for k in range(4)}
        labels.append(int(evaluate_node(tree, env)))
    return np.asarray(labels, dtype=int)


def build_cube():
    X = np.array(list(itertools.product((0, 1), repeat=4)), dtype=int)
    edges = [(i, j) for i in range(16) for j in range(i + 1, 16)
             if np.count_nonzero(X[i] != X[j]) == 1]
    assert X.shape == (16, 4) and len(edges) == 32
    assert np.allclose(np.linalg.norm(BASIS, axis=1), 1.0)
    assert len(np.unique(np.round(X @ BASIS, 9), axis=0)) == 16
    return X, edges


def boundary_edges(edges, y):
    return [(i, j) for i, j in edges if y[i] != y[j]]


def augmented(X):
    return np.column_stack((np.asarray(X, dtype=float), np.ones(len(X))))
```

```
def linear_separability(X, y):
    t = 2 * y - 1
    A = augmented(X)
    kw = dict(c=np.zeros(5), A_ub=-(t[:, None] * A),
              b_ub=-np.ones(len(X)), bounds=[(None, None)] * 5)
    try:
        r = linprog(method='highs', **kw)
    except ValueError:
        r = linprog(method='interior-point', **kw)
    if r.status == 2:
        return False, None
    if not r.success:
        raise RuntimeError('Проверка разделимости: ' + str(r.message))
    if np.min(t * (A @ r.x)) < 1 - 1e-6:
        raise RuntimeError('Нарушены ограничения LP из-за численной ошибки')
    return True, np.asarray(r.x, dtype=float)


def predict_threshold(X, theta):
    return (augmented(X) @ theta >= 0).astype(int)


def train_perceptron(X, y, eta=ETA, max_epochs=MAX_EPOCHS):


    w = np.array([0., 0., 0., 0., W0_INITIAL])
    A = augmented(X)
    history = []
    for epoch in range(1, max_epochs + 1):
        updates = 0
        for row, expected in zip(A, y):
            actual = int(np.dot(w, row) >= 0)
            delta = int(expected) - actual
            if delta:
                w += eta * delta * row
                updates += 1
        mistakes = int(np.count_nonzero(predict_threshold(X, w) != y))
        history.append((epoch, updates, mistakes))
        if mistakes == 0:
            return w, history, True
    return w, history, False


def train_fisher(X, y):
    c0 = X[y == 0].astype(float)
    c1 = X[y == 1].astype(float)
    m0, m1 = c0.mean(axis=0), c1.mean(axis=0)
    d0, d1 = c0 - m0, c1 - m1
    S0, S1 = d0.T @ d0, d1.T @ d1
    SW = S0 + S1
    det = float(np.linalg.det(SW))
    inverse = np.linalg.inv(SW) if abs(det) > 1e-10 else np.linalg.pinv(SW)
    raw = inverse @ (m1 - m0)
```

```
    norm = float(np.linalg.norm(raw))
    if norm <= 1e-12:
        raise ValueError('Направление Фишера не определено')
    w = raw / norm
    z = X @ w
    z0, z1 = z[y == 0], z[y == 1]
    threshold = float((z0.max() + z1.min()) / 2)
    means = (float(z0.mean()), float(z1.mean()))
    variances = (float(np.sum((z0 - means[0])**2)), float(np.sum((z1 - means[1])**2)))
    J = (means[1]-means[0])**2/sum(variances)
    details = dict(m0=m0,m1=m1,S0=S0,S1=S1,SW=SW,inverse=inverse,
                   raw=raw, w=w, projected=z, z0=z0, z1=z1,
                   projmeans=means, variances=variances, between=(means[1]-means[0])**2,
                   within=sum(variances),J=J,threshold=threshold,det=det,
                   overlap=bool(z0.max() >= z1.min()))
    return np.r_[w,-threshold],details


def fisher_criterion(theta, SW, delta):
    w = theta[:4]
    num = float((w @ delta)**2)
    den = float(w @ SW @ w)
    if den <= 1e-14:
        return float('nan') if num <= 1e-14 else float('inf')
    return num/den


def train_simplex(X, y, boundary):
    boundary_indices = sorted({k for edge in boundary for k in edge})
    candidate = np.mean(X[y == 1],axis=0) - np.mean(X[y == 0],axis=0)
    preferred = tuple(int(v < 0) for v in candidate)
    flips = [preferred] + [x for x in itertools.product([0,1],repeat=4) if x != preferred]
    for flip in flips:
        mask = np.asarray(flip,dtype=bool)
        Xt = X.astype(float).copy()
        Xt[:,mask] = 1-Xt[:,mask]
        rows = np.array([np.r_[Xt[i] if y[i] == 0 else -Xt[i], 1]
                         for i in boundary_indices])
        rhs = np.array([1. if y[i] == 0 else -1. for i in boundary_indices])
        result = linprog([0,0,0,0,-1],A_ub=rows,b_ub=rhs,
                         bounds=[(0,None)]*5,method='highs-ds')
        if not result.success:
            continue
        a = result.x[:4]
        lam = float(result.x[4])
        w = np.where(mask,-a,a)
        b = float(sum(a[mask])-1)
        scores = X@w+b
        signed = (2*y-1)*scores
        if lam < -1e-7 or np.min(signed) < -1e-7:
            continue
        details = dict(flip=mask,XT=Xt, a=a,lam=lam,solution=result.x,
                       boundary_indices=boundary_indices,rows=rows,rhs=rhs,
```

```
                        constraints_count=len(boundary_indices),
                        all_constraints_ok=bool(np.min(signed)>=-1e-7))
        return np.r_[w,b],details
    raise ValueError('Не удалось решить задачу симплекс-методом после преобразования координат')


def unit_signed_margin(theta, X, y):
    normal = np.linalg.norm(theta[:4])
    if normal < 1e-12:
        return float('nan')
    return float(np.min((2*y - 1) * (augmented(X) @ theta) / normal))


def oriented_angle_degrees(a, b):
    x, z = a[:4], b[:4]
    val = float(np.dot(x, z) / (np.linalg.norm(x)*np.linalg.norm(z)))
    return float(np.degrees(np.arccos(np.clip(val, -1.0, 1.0))))


def boundary_intersections(X, edges, y, theta):
    scores = augmented(X) @ theta
    result = {}
    for i, j in edges:
        if y[i] == y[j]:
            continue
        z, o = (i, j) if y[i] == 0 else (j, i)
        g0, g1 = scores[z], scores[o]
        if abs(g1 - g0) <= 1e-12:
            frac = 0.0 if abs(g0) <= 1e-12 else None
        else:
            frac = float(-g0/(g1-g0))
            if frac < -1e-7 or frac > 1+1e-7:
                frac = None
            elif frac is not None:
                frac = float(np.clip(frac, 0, 1))
        result[(i,j)] = frac
    return result


def section_vertices(X, edges, theta):
    values = augmented(X) @ theta
    points = []
    for i, j in edges:
        a, b = float(values[i]), float(values[j])
        if abs(a) <= EPS:
            points.append(X[i].astype(float))
        if abs(b) <= EPS:
            points.append(X[j].astype(float))
        if a*b < 0:
            frac = -a/(b-a)
            points.append(X[i] + frac * (X[j] - X[i]))
    if not points:
        return np.empty((0,4), dtype=float)
```

```
    values = np.array(points)
    _, indices = np.unique(np.round(values, 9), axis=0, return_index=True)
    return values[np.sort(indices)]


def make_section_artists(ax, X, edges, theta, color):


    points4 = section_vertices(X, edges, theta)
    if len(points4) == 0:
        return [], points4
    Q = points4 @ BASIS
    artists = []
    centered = Q - Q.mean(axis=0)
    singular_values = np.linalg.svd(centered, compute_uv=False)
    rank = int(np.sum(singular_values > 1e-8))
    if rank == 3 and len(Q) >= 4:
        try:
            faces = [Q[tri] for tri in ConvexHull(Q).simplices]
            surface = Poly3DCollection(faces, facecolor=color, edgecolor=color,
                                       alpha=0.13, linewidths=0.65)
            ax.add_collection3d(surface)
            artists.append(surface)
        except QhullError:
            pass
    elif rank == 2 and len(Q) >= 3:
        _, _, vh = np.linalg.svd(centered, full_matrices=False)
        hull2 = ConvexHull(centered @ vh[:2].T)
        poly = Poly3DCollection([Q[hull2.vertices]], facecolor=color,
                                edgecolor=color, alpha=0.28)
        ax.add_collection3d(poly)
        artists.append(poly)
    elif rank == 1 and len(Q) >= 2:
        d = np.linalg.svd(centered, full_matrices=False)[2][0]
        ordered = Q[np.argsort(centered @ d)]
        artists += ax.plot(ordered[[0,-1], 0], ordered[[0,-1], 1],
                           ordered[[0,-1], 2], color=color, linewidth=3)
    cloud = ax.scatter(Q[:,0], Q[:,1], Q[:,2], s=32, c=color,
                       marker='x', linewidths=1.7, depthshade=False)
    artists.append(cloud)
    return artists, points4


def format_eq(theta):
    return ' '.join('%+.6f*x%d' % (theta[i], i+1) for i in range(4)) + ' %+.6f = 0' % theta[4]


def show_table(X, y):
    print('\nТАБЛИЦА ИСТИННОСТИ (все 16 вершин)')
    print_grid(['x1', 'x2', 'x3', 'x4', 'f'],
               [list(map(int, bits)) + [int(v)] for bits, v in zip(X, y)])


```

```
314 def save_table(path, X, y):
315     np.savetxt(str(path), np.column_stack((X,y)), delimiter=',', fmt='%d',
316                header='x1,x2,x3,x4,f', comments='', encoding='utf-8')
317
318
319 def print_grid(headers, rows, alignments=None):
320     rows = [[str(v) for v in row] for row in rows]
321     headers = [str(h) for h in headers]
322     widths = [max(len(headers[j]), *(len(row[j]) for row in rows)) + 2
323               for j in range(len(headers))]
324     horizontal = '+' + '+'.join('-' * w for w in widths) + '+'
325
326     def format_row(row):
327         return '|' + '|'.join(' ' + cell.ljust(widths[j] - 1)
328                               for j, cell in enumerate(row)) + '|'
329
330     print(horizontal)
331     print(format_row(headers))
332     print(horizontal)
333     for row in rows:
334         print(format_row(row))
335     print(horizontal)
336
337
338 def print_models(X, y, edges, models, stats, boundary):
339     print('\nСРАВНЕНИЕ МЕТОДОВ (16 вершин)')
340     print_grid(['Метод','Верно','Ошибки','Мин. зазор','Критерий J','Гран. рёбра'],
341                [[name,'%d/16'%v['correct'],v['errors'],'%.5f'%v['margin'],
342                  '%.6f'%v['J'],'%d/%d'%(v['boundary_cross'],len(boundary))]
343                 for name,v in stats.items()])
344     print('\nКОЭФФИЦИЕНТЫ ГИПЕРПЛОСКОСТЕЙ')
345     print_grid(['Метод','w1','w2','w3','w4','b'],
346                [[name]+['%.6f'%v for v in theta] for name,theta in models.items()])
347     print('\nУРАВНЕНИЯ')
348     for name,theta in models.items():
349         print('%-12s %s'%(name,format_eq(theta)))
350     print('\nУГЛЫ ОТНОСИТЕЛЬНО ФИШЕРА')
351     print_grid(['Метод','Угол, °'],
352                [[name,'%.4f'%oriented_angle_degrees(theta,models['Фишер'])]
353                 for name,theta in models.items()])
354     print('\nПЕРЕСЕЧЕНИЯ ГРАНИЧНЫХ РЁБЕР (от класса 0 к классу 1)')
355     fracs={name:boundary_intersections(X,edges,y,theta) for name,theta in models.items()}
356     rows=[]
357     for i,j in boundary:
358         edge='%s -- %s'%(''.join(map(str,X[i])),''.join(map(str,X[j])))
359         rows.append([edge]+['%.6f'%fracs[name][(i,j)] if fracs[name][(i,j)] is not None else
        ↪  'нет'
360                             for name in models])
361     print_grid(['Ребро']+list(models),rows)
362
363
364 def print_fisher_details(d):
365     print('\nФИШЕР: ЦЕНТРОИДЫ')
```

```
    print_grid(['Класс','x1','x2','x3','x4'],
               [[label]+['%.6f'%v for v in d['m'+label]] for label in ('0','1')])
    for key in ('S0','S1','SW','inverse'):
        print('\n'+key)
        print_grid(['']+['x%d'%j for j in range(1,5)],
                   [['x%d'%(i+1)]+['%.6f'%v for v in row]
                    for i,row in enumerate(d[key])])
    print('\nФИШЕР: ПРОЕКЦИИ')
    print_grid(['Класс','Точки проекции'],
               [[str(c),' '.join('%.5f'%v for v in d['z'+str(c)])] for c in (0,1)])
    print_grid(['Показатель','Значение'],[
        ['||w||','%.6f'%np.linalg.norm(d['w'])],
        ['Средняя проекция 0','%.6f'%d['projmeans'][0]],
        ['Средняя проекция 1','%.6f'%d['projmeans'][1]],
        ['Разброс между классами','%.6f'%d['between']],
        ['Разброс внутри класса 0','%.6f'%d['variances'][0]],
        ['Разброс внутри класса 1','%.6f'%d['variances'][1]],
        ['Суммарный разброс','%.6f'%d['within']],
        ['Критерий J','%.6f'%d['J']],
        ['Порог крайних проекций','%.6f'%d['threshold']]])


def print_simplex_details(d):
    print('\nСИМПЛЕКС: ПРЕОБРАЗОВАНИЕ КООРДИНАТ')
    print_grid(['Координата','Преобразование','lambda_j'],[
        ['x%d'%(i+1),'1 - x%d'%(i+1) if d['flip'][i] else 'x%d'%(i+1),
         '%.6f'%d['a'][i]] for i in range(4)])
    print_grid(['Показатель','Значение'],[
        ['Нормировка порога','1'],['Максимальный алгебраический отступ','%.6f'%d['lam']],
        ['Граничных вершин в ограничениях',str(d['constraints_count'])]])


def save_projection(path, d):
    fig,ax=plt.subplots(figsize=(10,3.7))
    zz=d['projected']
    for c,col in ((0,COLOR_ZERO),(1,COLOR_ONE)):
        inds=np.where(d['labels']==c)[0]
        ax.scatter(zz[inds],np.full(len(inds),c),s=65,color=col,
                   edgecolor='white',linewidth=0.5,label='Класс '+str(c))
        for i in inds:
            ax.annotate(''.join(map(str,d['X'][i])),(zz[i],c),xytext=(0,9),
                        textcoords='offset points',ha='center',fontsize=7)
    ax.axvline(d['threshold'],ls='--',lw=1.8,color='#d18c19',label='Порог Фишера')
    ax.set_xlabel('Проекция w^T x')
    ax.set_yticks([0,1]);ax.set_ylim(-0.33,1.42)
    ax.grid(axis='x',alpha=0.2)
    ax.legend(loc='lower right',ncol=3,fontsize=8)
    fig.tight_layout()
    fig.savefig(str(path),dpi=180)
    plt.close(fig)


def save_stats(path, stats):
```

```
    with open(str(path), 'w', encoding='utf-8-sig', newline='') as fp:
        writer = csv.writer(fp, delimiter=';')
        writer.writerow(['method', 'correct_of_16', 'errors', 'accuracy_pct',
                         'signed_min_margin', 'fisher_J', 'boundary_edges_crossed',
                         'boundary_edges_total', 'epochs', 'w1', 'w2', 'w3', 'w4', 'b'])
        for name, s in stats.items():
            writer.writerow([name, s['correct'], s['errors'],
                             100*s['correct']/16, s['margin'], s['J'],
                             s['boundary_cross'], s['boundary_total'], s['epochs'],
                             *s['weights']])


def show_hypercube(X, y, edges, boundary, models, stats, expression, image_path, show=True,
↪  view='perceptron'):
    P = X @ BASIS
    fig = plt.figure(figsize=(16, 9.8))
    ax = fig.add_axes([0.025, 0.075, 0.71, 0.855], projection='3d')
    ax.set_axis_off()
    if hasattr(ax, 'set_proj_type'):
        ax.set_proj_type('ortho')

    bound_set = set(boundary)
    for i, j in edges:
        pts = P[[i, j]]
        bnd = (i, j) in bound_set
        along_fourth = X[i, 3] != X[j, 3]
        ax.plot(pts[:, 0], pts[:, 1], pts[:, 2],
                color=COLOR_BOUNDARY if bnd else '#858e99',
                linewidth=2.65 if bnd else 1.05,
                linestyle='--' if along_fourth else '-',
                alpha=0.92 if bnd else 0.62)

    plane_artists = {}
    state = {name: view == 'all' or (view == 'fisher' and name == 'Фишер') or (view == 'simplex'
    ↪  and name == 'Симплекс') or (view == 'perceptron' and name == 'Перцептрон') for name in
    ↪  models}
    for name, theta in models.items():
        objs, _ = make_section_artists(ax, X, edges, theta, COLORS[name])
        plane_artists[name] = objs
        for artist in objs:
            artist.set_visible(state[name])

    for k, direction in enumerate(BASIS):
        tip = direction * 1.48
        ax.quiver(0, 0, 0, *tip, color='#20242a',
                  arrow_length_ratio=0.085, linewidth=2.1)
        ax.text(*(direction * 1.70), 'x%d' % (k + 1), fontsize=13.5,
                weight='bold', color='#20242a')

    for xyz, bits, label in zip(P, X, y):
        ax.scatter(*xyz, color=COLOR_ONE if label else COLOR_ZERO,
                   s=81, edgecolors='white', linewidths=0.8,
                   depthshade=False)
```

```
        text = '(' + ','.join(map(str, bits)) + ')'
        ax.text(*(xyz + np.array([0.025, 0.03, 0.025])),
                text, fontsize=8.3, color='#20242a')

    ax.view_init(elev=22, azim=38)
    visible = np.vstack((P, BASIS * 1.78, np.zeros((1, 3))))
    middle = 0.5 * (visible.max(axis=0) + visible.min(axis=0))
    radius = 0.55 * np.max(visible.max(axis=0) - visible.min(axis=0))
    ax.set_xlim(middle[0] - radius, middle[0] + radius)
    ax.set_ylim(middle[1] - radius, middle[1] + radius)
    ax.set_zlim(middle[2] - radius, middle[2] + radius)
    if hasattr(ax, 'set_box_aspect'):
        try:
            ax.set_box_aspect((1, 1, 1), zoom=1.15)
        except TypeError:
            ax.set_box_aspect((1, 1, 1))

    fig.suptitle('4D-гиперкуб: перцептрон, Фишер, симплекс-метод', y=0.978,
                 fontsize=15)
    fig.text(0.035, 0.926,
             'Граничные рёбра выделены красным; координаты вершин -- (x1,x2,x3,x4)',
             fontsize=10)
    legend = [
        Line2D([0], [0], marker='o', markersize=9, markerfacecolor=COLOR_ZERO,
               markeredgecolor='white', color='none', label='Класс f=0'),
        Line2D([0], [0], marker='o', markersize=9, markerfacecolor=COLOR_ONE,
               markeredgecolor='white', color='none', label='Класс f=1'),
        Line2D([0], [0], color=COLOR_BOUNDARY, linewidth=2.5,
               label='Граничное ребро'),
    ]
    fig.legend(handles=legend, loc='upper left', bbox_to_anchor=(0.035, 0.895),
               fontsize=9)

    names = list(models)
    if names:
        widget_ax = fig.add_axes([0.763, 0.72, 0.22, 0.182])
        widget_ax.set_title('Показывать:', loc='left', fontsize=11)
        labels = names
        checks = CheckButtons(widget_ax, labels, [state[n] for n in labels])
        for label in checks.labels:
            label.set_fontsize(10)

        def toggle(name):
            state[name] = not state[name]
            if name in plane_artists:
                for artist in plane_artists[name]:
                    artist.set_visible(state[name])
            fig.canvas.draw_idle()

        checks.on_clicked(toggle)
        fig._toggle_reference = checks

        fig.text(0.767, 0.683, 'Гиперплоскости', fontsize=10, weight='bold')
```

```
        for pos, name in enumerate(names):
            fig.text(0.772, 0.654 - pos * 0.033, '●  ' + name,
                     fontsize=10, color=COLORS[name])

        fig.text(0.767, 0.535, 'Сравнение на 16 вершинах',
                 fontsize=11, weight='bold')
        table_ax = fig.add_axes([0.755, 0.24, 0.238, 0.277])
        table_ax.set_axis_off()
        rows = [
            ['Верно'] + ['%d/16' % stats[n]['correct'] for n in names],
            ['Ошибки'] + [str(stats[n]['errors']) for n in names],
            ['Зазор'] + ['%.4f' % stats[n]['margin'] for n in names],
            ['Фишер J'] + ['%.4f' % stats[n]['J'] for n in names],
            ['Гран. рёбра'] + [
                '%d/%d' % (stats[n]['boundary_cross'], stats[n]['boundary_total'])
                for n in names
            ],
        ]
        headers = ['Показатель', 'Перцеп.', 'Фишер', 'ЛП']
        table = table_ax.table(cellText=rows, colLabels=headers,
                               colWidths=[0.37, 0.22, 0.215, 0.195],
                               cellLoc='center', colLoc='center',
                               bbox=[0, 0, 1, 1])
        table.auto_set_font_size(False)
        table.set_fontsize(8.5)
        for (row, col), cell in table.get_celld().items():
            cell.set_edgecolor('#d1d5db')
            cell.set_linewidth(0.65)
            cell.set_facecolor('#edf2f8' if row == 0 else
                               '#f6f8fb' if row % 2 == 0 else '#ffffff')
            cell.set_text_props(weight='bold' if row == 0 else 'normal')
            if col == 0 and row > 0:
                cell.get_text().set_ha('left')
                cell.PAD = 0.065
        fig.text(0.766, 0.209,
                 'ЛП: линейное программирование, симплекс-метод.',
                 fontsize=8.3, color='#64748b', va='top')
    else:
        fig.text(0.767, 0.70, 'Нет разделяющих плоскостей',
                 fontsize=10.5, weight='bold')

    fig.text(0.5, 0.027,
             'Вращайте мышью; флажками справа включайте гиперплоскости.',
             ha='center', fontsize=9)
    if image_path:
        fig.savefig(str(image_path), dpi=175)
        print('График сохранён:', image_path)
    if show:
        plt.show()
    else:
        plt.close(fig)
    return plane_artists

```

```
575
576 def run(expression, truth_csv='truth_table_4d.csv',
577         metrics_csv='comparison_4d.csv', image_path='hypercube_comparison.png',
578         projection_path='fisher_projections.png', show=True,view='perceptron'):
579     X,edges=build_cube()
580     y=function_labels(expression,X)
581     show_table(X,y)
582     save_table(truth_csv,X,y)
583     boundary=boundary_edges(edges,y)
584     print('\nВершин:',len(X),'| рёбер:',len(edges),'| граничных:',len(boundary))
585     print_grid(['Ребро','Изменяется координата'],
586                [['%s -- %s'%(''.join(map(str,X[i])),''.join(map(str,X[j]))),
587                  'x%d'%(int(np.flatnonzero(X[i]!=X[j])[0])+1)] for i,j in boundary])
588     if len(np.unique(y))==1:
589         print('Один класс: две дискриминантные группы отсутствуют.')
590         save_stats(metrics_csv,{})
591         show_hypercube(X,y,edges,boundary,{}, {},expression,image_path,show,view)
592         return dict(X=X,y=y,edges=edges,boundary=boundary,separable=False,models={})
593     separable,witness=linear_separability(X,y)
594     print('\nСТРОГАЯ ЛИНЕЙНАЯ РАЗДЕЛИМОСТЬ:', 'ДА' if separable else 'НЕТ')
595     if not separable:
596         print('Нельзя построить безошибочное линейное разделение.')
597         save_stats(metrics_csv,{})
598         show_hypercube(X,y,edges,boundary,{}, {},expression,image_path,show,view)
599         return dict(X=X,y=y,edges=edges,boundary=boundary,separable=False,models={})
600     perceptron,history,ok=train_perceptron(X,y)
601     if not ok:
602         raise RuntimeError('Перцептрон не сошёлся за %d эпох'%MAX_EPOCHS)
603     fisher,fd=train_fisher(X,y)
604     simplex,ld=train_simplex(X,y,boundary)
605     models={'Перцептрон':perceptron,'Фишер':fisher,'Симплекс':simplex}
606     stats={}
607     for name,theta in models.items():
608         pred=predict_threshold(X,theta)
609         err=int(np.count_nonzero(pred!=y))
610         cr=boundary_intersections(X,edges,y,theta)
611         stats[name]=dict(correct=len(X)-err,errors=err,margin=unit_signed_margin(theta,X,y),
612                          J=fisher_criterion(theta,fd['SW'],fd['m1']-fd['m0']),
613                          boundary_cross=sum(v is not None for v in cr.values()),
614                          boundary_total=len(boundary),
615                          epochs=len(history) if name=='Перцептрон' else None,
616                          weights=theta.tolist())
617     print('\nПерцептрон: пороговая активация, eta=%.2f, эпох %d'%(ETA,len(history)))
618     print_fisher_details(fd)
619     print_simplex_details(ld)
620     print_models(X,y,edges,models,stats,boundary)
621     save_stats(metrics_csv,stats)
622     fd['X']=X
623     fd['labels']=y
624     save_projection(projection_path,fd)
625     show_hypercube(X,y,edges,boundary,models,stats,expression,image_path,show,view)
626     return dict(X=X,y=y,edges=edges,boundary=boundary,separable=True,
627                 models=models,stats=stats,history=history,fisher=fd,simplex=ld)
```

```
628
629
630 def main():
631     parser=argparse.ArgumentParser(description='Классификация 4D-гиперкуба: перцептрон, Фишер,
        ↪ симплекс')
632     parser.add_argument('--expr',help='Булева функция от x1,x2,x3,x4')
633     parser.add_argument('--no-show',action='store_true')
634     parser.add_argument('--view',choices=['perceptron','fisher','simplex','all'],default='percep
        ↪ tron')
635     parser.add_argument('--truth-csv',default='truth_table_4d.csv')
636     parser.add_argument('--metrics-csv',default='comparison_4d.csv')
637     parser.add_argument('--png',default='hypercube_comparison.png')
638     parser.add_argument('--projection-png',default='fisher_projections.png')
639     args=parser.parse_args()
640     expr=args.expr
641     if expr is None:
642         print('Введите булеву функцию (and, or, not, &, |, ~, ^, ¬, ∧, ∨).')
643         print('Вариант 2:',EXAMPLE_FUNCTION)
644         try:
645             expr=input('f(x1,x2,x3,x4) = ').strip() or EXAMPLE_FUNCTION
646         except EOFError:
647             expr=EXAMPLE_FUNCTION
648     try:
649         run(expr,truth_csv=Path(args.truth_csv),metrics_csv=Path(args.metrics_csv),
650             image_path=Path(args.png),projection_path=Path(args.projection_png),
651             show=not args.no_show,view=args.view)
652     except (SyntaxError,ValueError) as exc:
653         parser.error('Некорректная функция или параметры: '+str(exc))
654
655
656 if __name__=='__main__':
657     main()
```